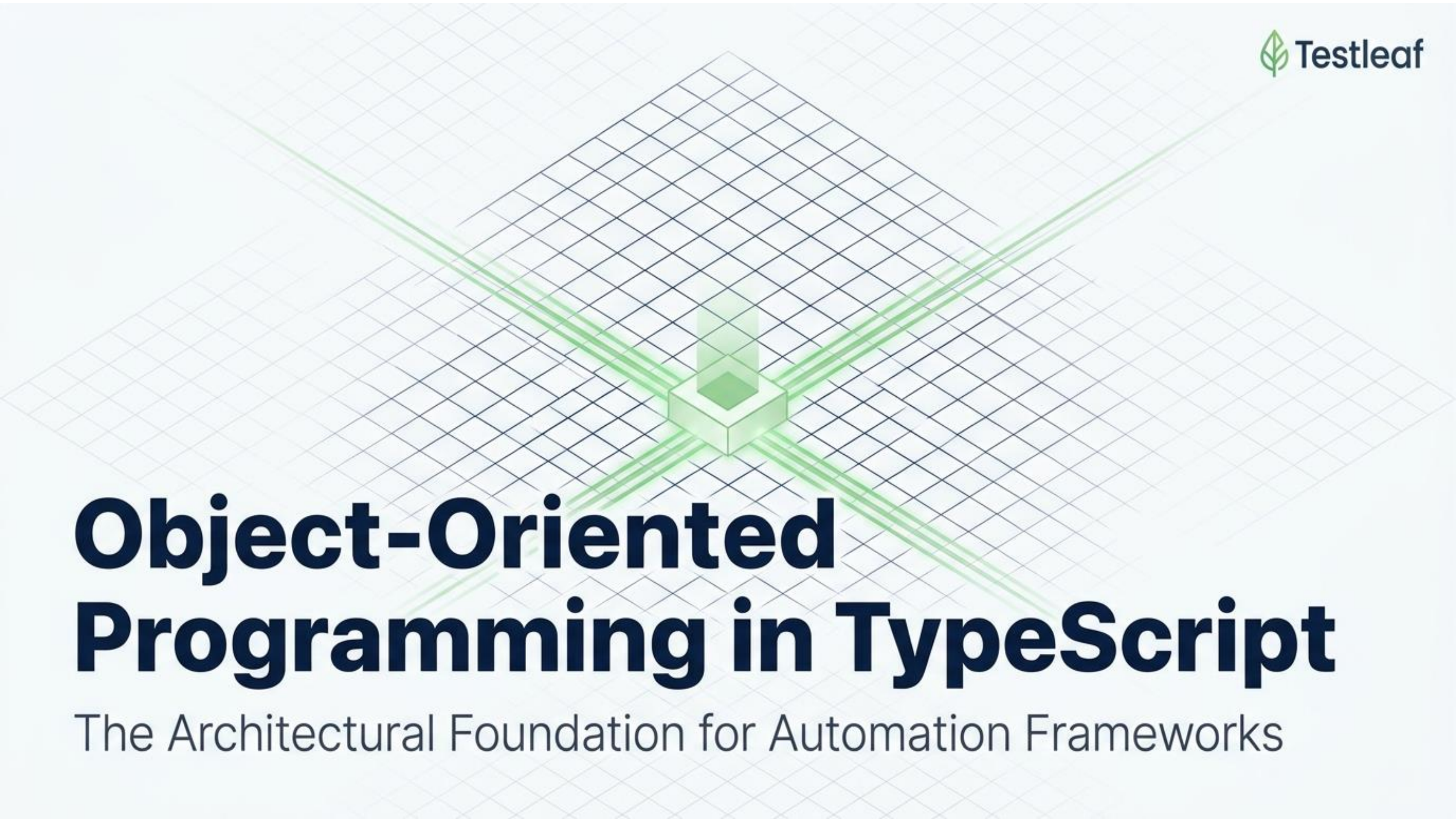
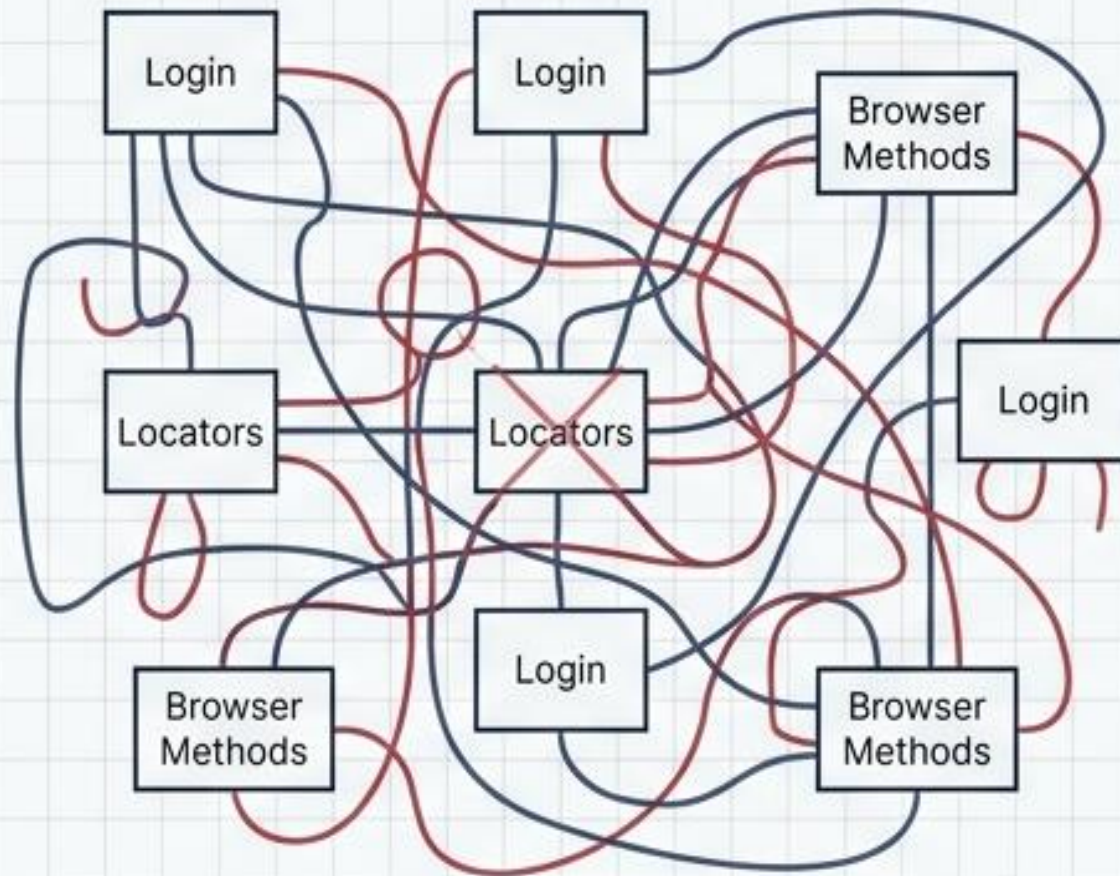




Object-Oriented Programming in TypeScript

The Architectural Foundation for Automation Frameworks

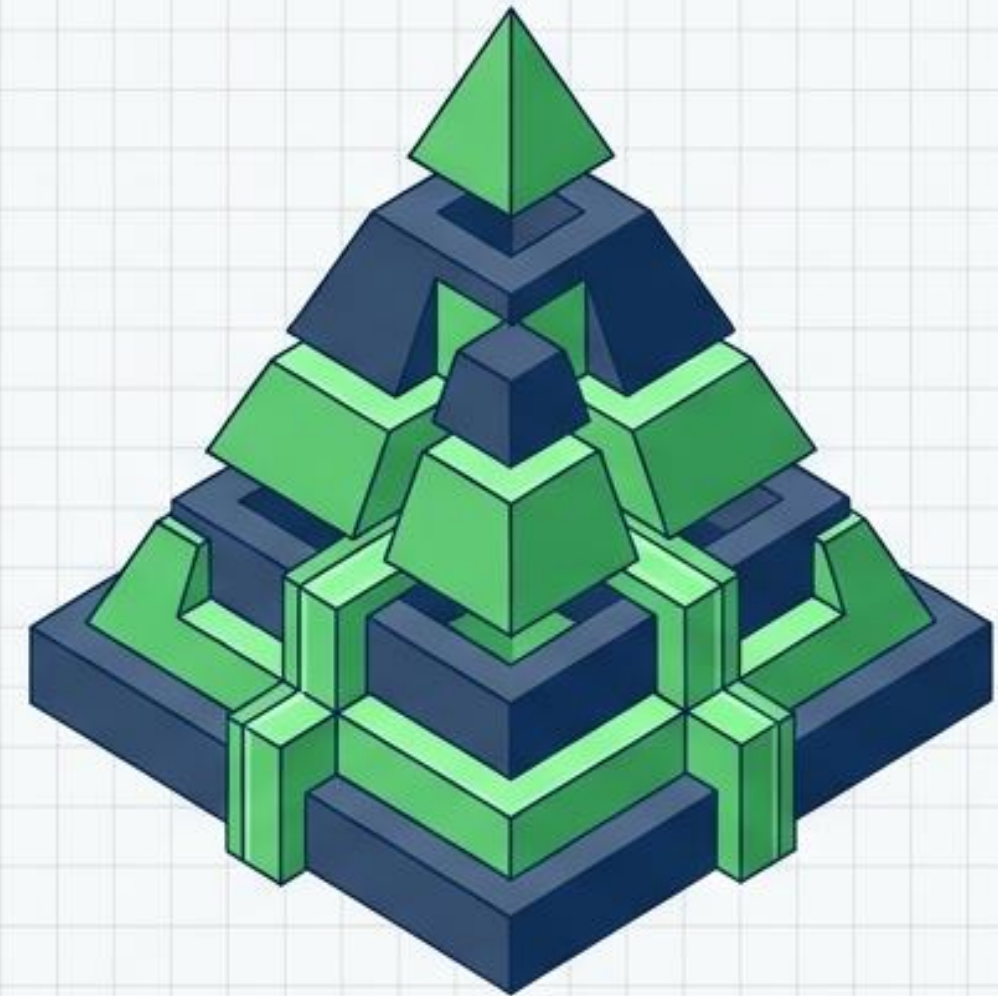
Without OOP



The Problem

- Copied 50 times
- High Maintenance
- Inconsistent Implementation

With OOP (Page Object Model)



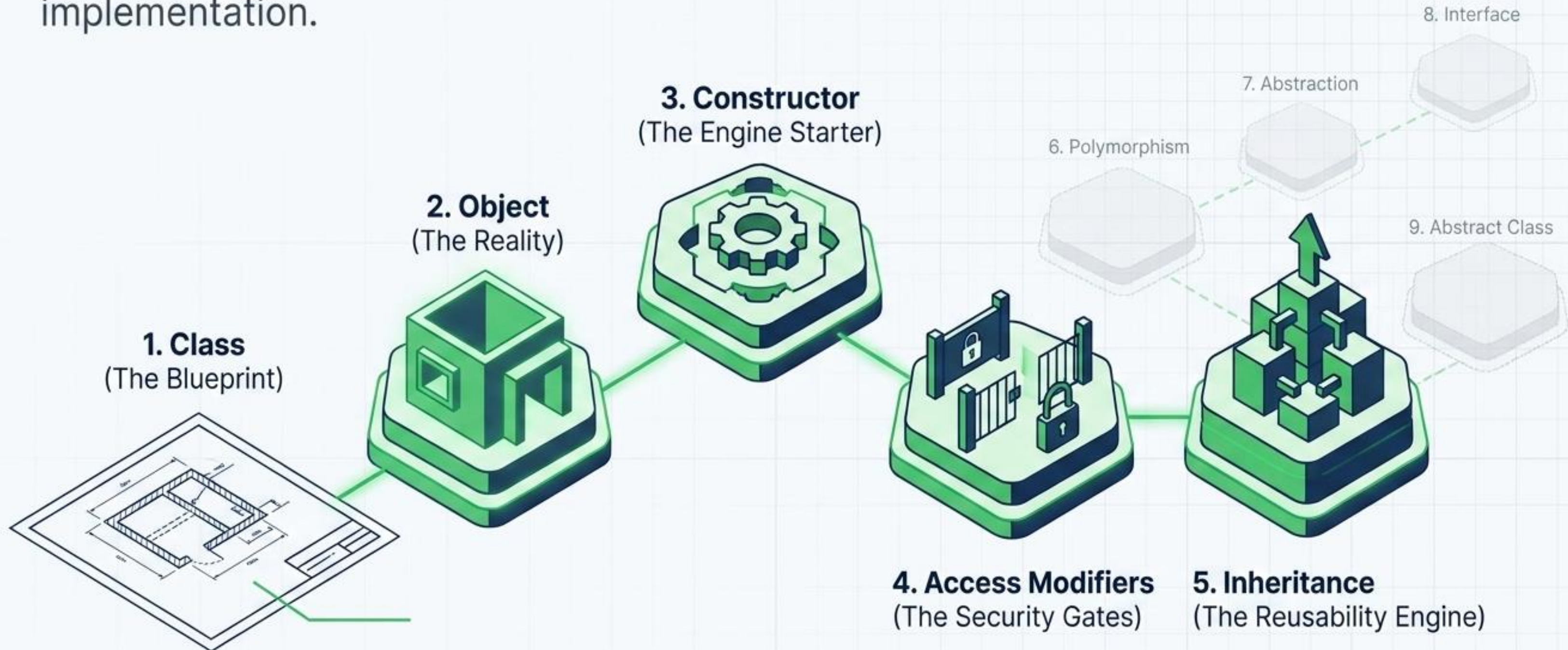
The Solution

- Code Reusable
- Industry Standard
- Easy Scaling

Today's Architectural Roadmap



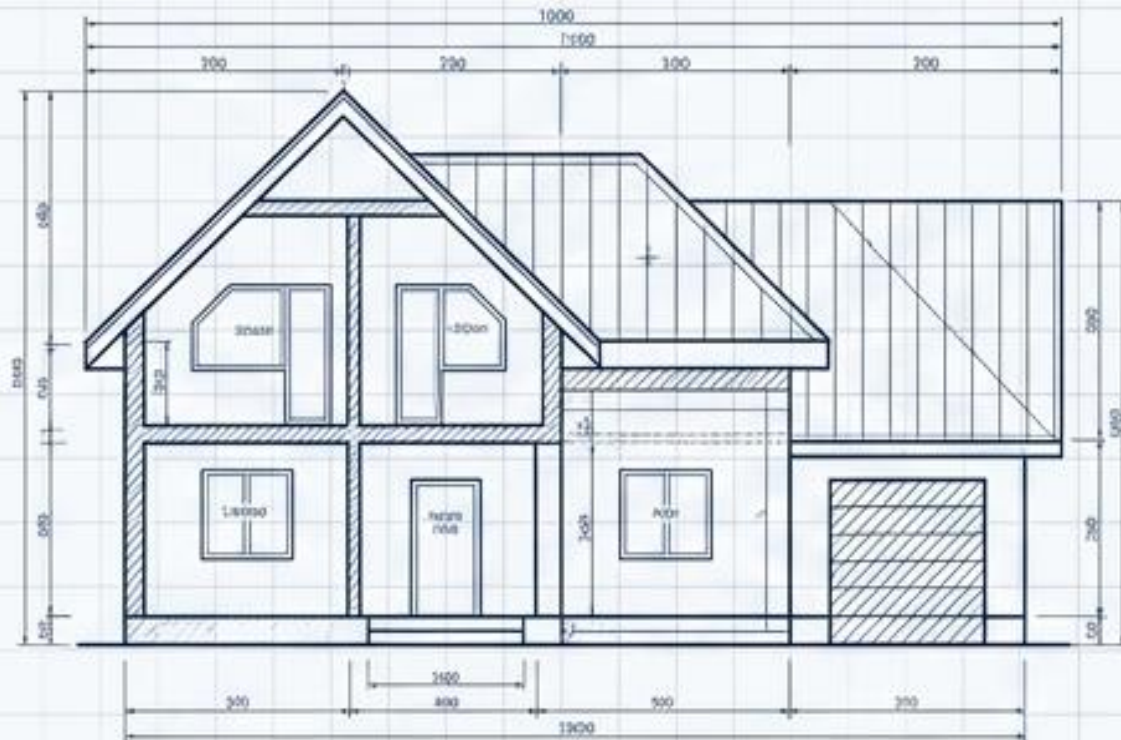
Moving from foundational design to scalable framework implementation.



1. Class

The Blueprint

Template / Design



CODE SNIPPET

```
class LoginPage {
  login() { console.log('Login Successful'); }
}
```

- DEFINES STRUCTURE

A Class is a blueprint or template used to create objects.

Crucial Context: At this stage, NO memory is allocated. No object is created. Only the abstract design exists.

2. Object

The Reality



Blueprint (LoginPage)

```
const page =  
new LoginPage();
```

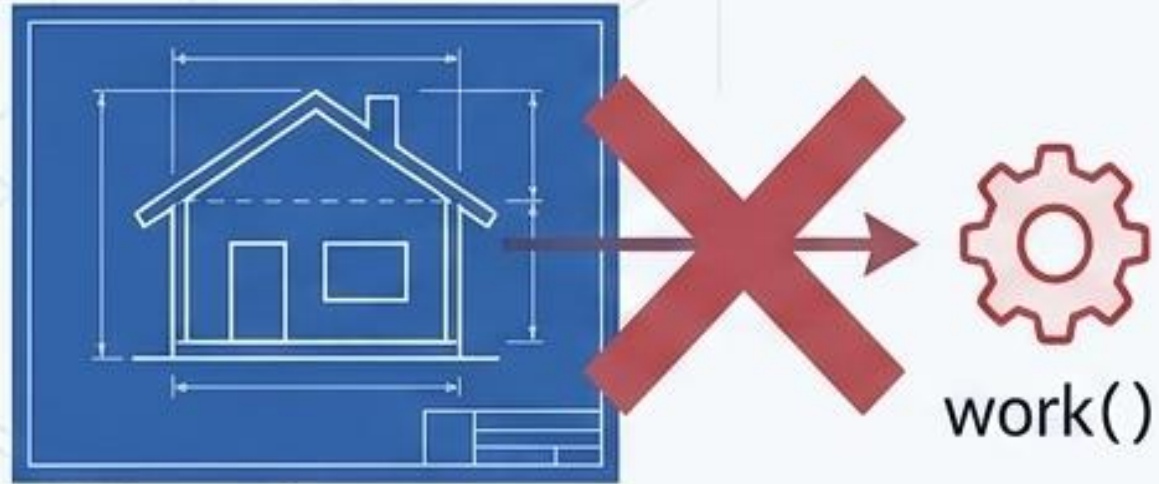


Memory Allocated!

An Object is an instance of a class. Memory is allocated **ONLY** when the 'new' keyword is executed.

Classes vs. Objects in Playwright

Why can't I call a method directly from the class?



Employee

(The Error)

```
Employee.work(); // ERROR
```

Class = Design.
You cannot drive a blueprint.

Playwright Context

```
// Real-Time Playwright Example  
  
const loginPage = new LoginPage();  
await loginPage.login();
```



emp

(The Correct Way)

```
const emp = new Employee();  
emp.work(); // SUCCESS
```

Object = Real Entity.
You must build it before using it.

3. Constructor

The Engine Starter



```
class Employee {  
  constructor() { console.log('Object Created'); }  
}
```



```
new Employee();
```

```
Auto-outputs: Object Created
```

Real-Time Playwright Context

Why use it? To initialize object data during creation.

```
class LoginPage { constructor(private page: Page) {} }
```

Result: The Playwright page object is instantly available throughout the entire class.

4. Access Modifiers

The Security Gates - Controlling Visibility



Deep Dive: The First Principle

Modifiers are SCOPE-BASED, not OBJECT-BASED.



Wrong Thinking:

- Object Created -> Therefore I can access everything.
- Object Ownership $\neq$ Access Permission

Correct Thinking:

- The compiler never asks 'Which object is being used?'
- It asks: "From which location is the call being made?"

The Access Modifier Matrix

Diagnostic Cheat Sheet

Location	public	private	protected
Same Class	✓ Allowed	✓ Allowed	✓ Allowed
Child Class	✓ Allowed	✗ Not Allowed	✓ Allowed
Outside Class	✓ Allowed	✗ Not Allowed	✗ Not Allowed

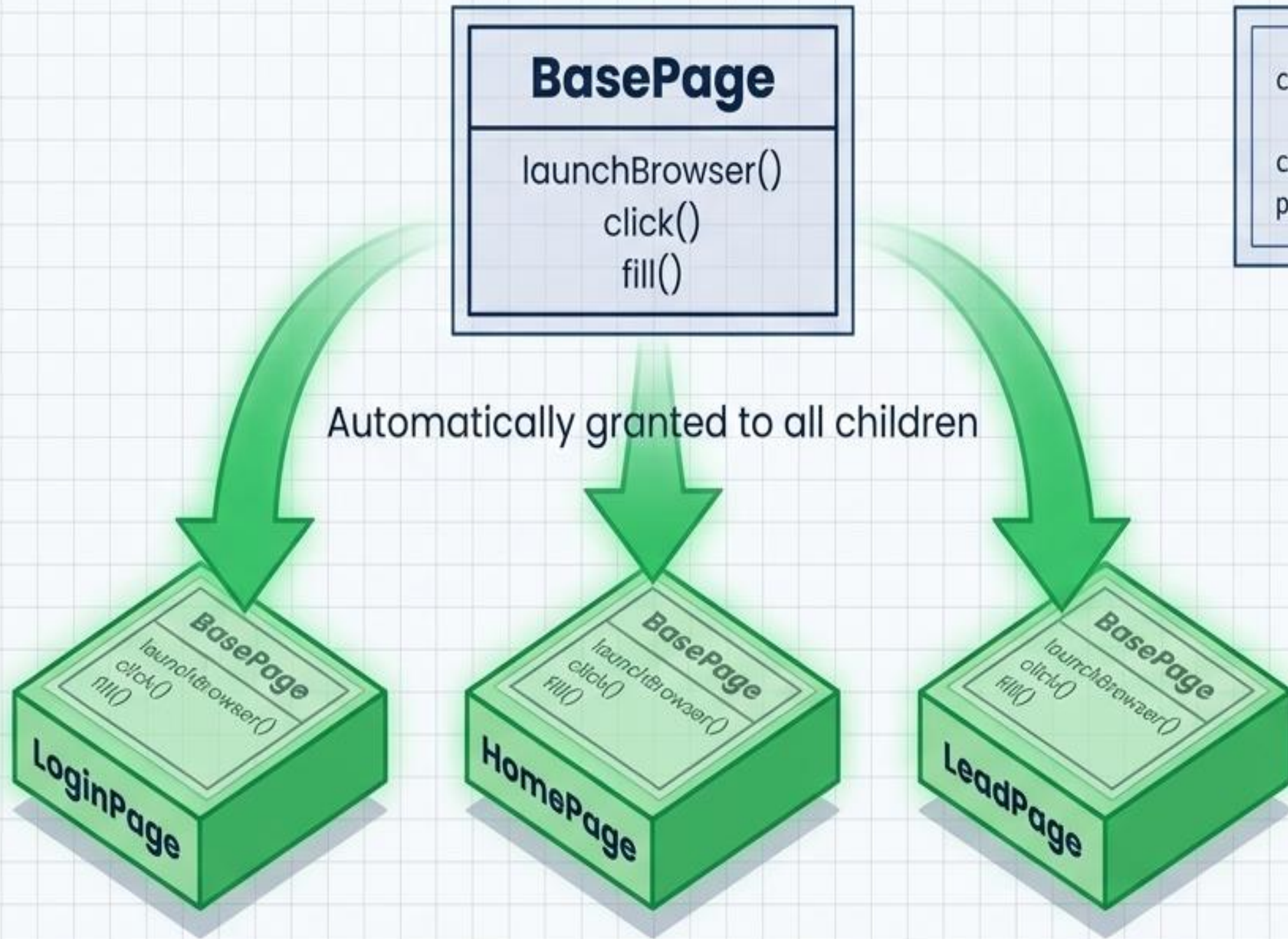


The Java Link: Both TypeScript and Java follow these exact same conceptual rules.

Difference: TS checks at compile-time for developer experience; Java enforces strictly at runtime/compile-time.

5. Inheritance

The Reusability Engine (Eliminating Duplication)

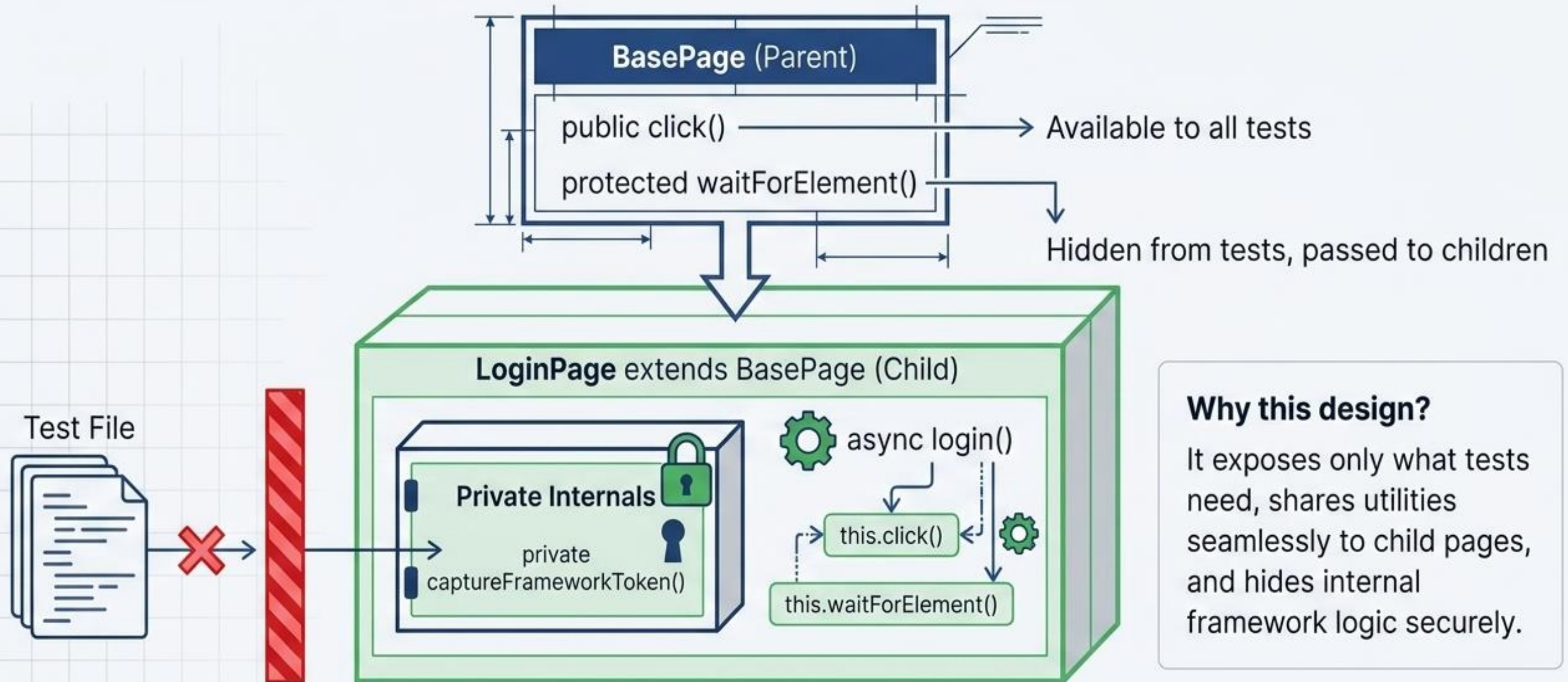


```
class LoginPage extends BasePage { ... }
```

```
const page = new LoginPage();  
page.launchBrowser(); // Inherited from BasePage
```

Real-World Architecture

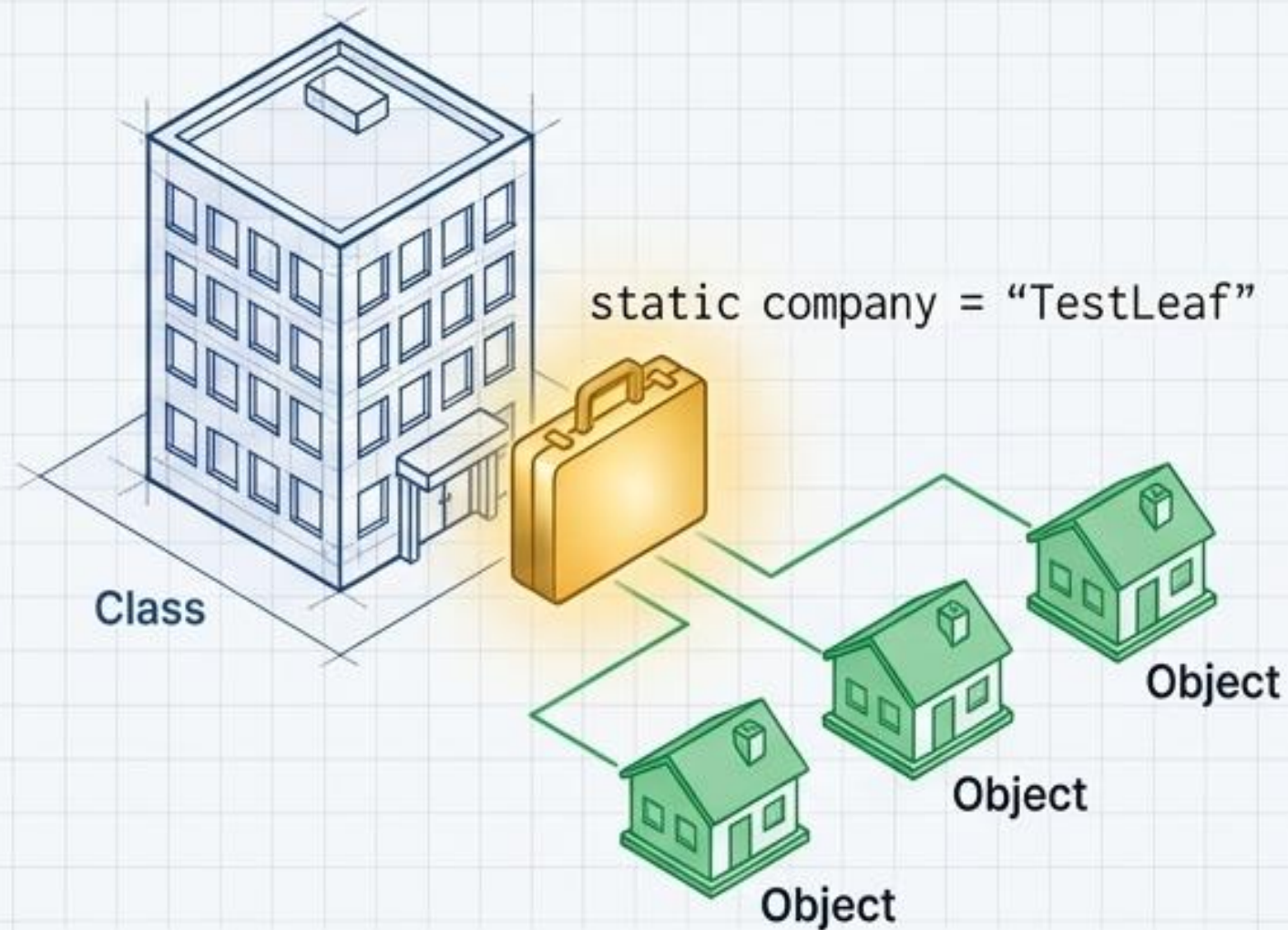
Modifiers and Inheritance in a Page Object Model



Edge Concepts

Static Variables & Methods vs. Functions

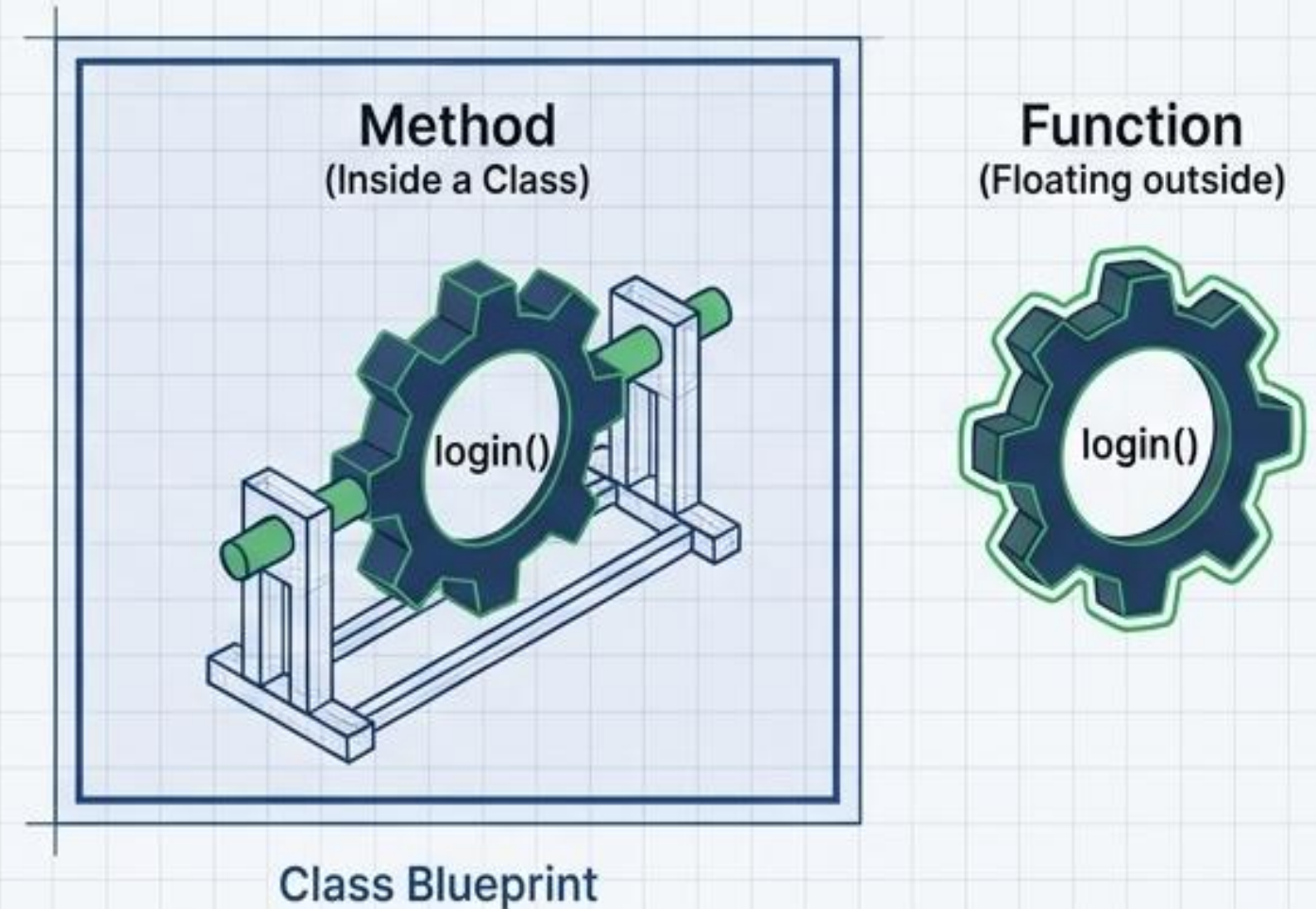
Static vs Instance



Static members belong to the Class itself,
not individual instances.

Called via: `Employee.payroll()`

Function vs Method



Every Method is a Function.
NOT every Function is a Method.

Key Takeaways

The Architectural Blueprint Mastered



Class: Blueprint / Template for creating objects.



Object: Actual instance of a class (memory allocated).



Constructor: Auto-executes on creation to initialize data.



Inheritance: Reuses functionality via the "extends" keyword.



public: Accessible everywhere (default).



private: Accessible only inside the exact same class.



protected: Accessible within the class + child classes.



Method vs Function: Method is bound to a class; Function is independent.



Static: Belongs to the class, no object required.

FINAL RULE: Access modifiers are scope-based, not object-based.
Ask **WHERE** the call is made, not **WHO** owns the object.